

Metodi avanzati di programmazione

Alessandro Carella

Indice

1	Astrazione nella progettazione	2
1.1	Astrazione funzionale	3
1.1.1	Specifiche semantiche	3
1.1.2	Limiti dell'astrazione funzionale	4
1.2	Astrazione dati	4
1.2.1	Information Hiding	4
1.2.2	Incapsulamento	5
1.3	Specifica sintattica e semantica	6
1.3.1	Specifiche assiomatiche	7
1.3.2	Specifiche algebriche	7
2	L'astrazione nella programmazione	13
2.1	Astrazione di funzione	14
2.1.1	Anomalie di progetto dei linguaggi	14
2.2	Astrazione di procedura	16

Capitolo 1

Astrazione nella progettazione

In ambito scientifico la parola "astrarre" significa cambiare la rappresentazione di un problema e questo serve a concentrarsi su aspetti *rilevanti* dimenticando gli elementi *incidentali*.

Il termine astrazione sotto-intende:

- un **processo**: l'estrazione delle informazioni essenziali e rilevanti per un particolare scopo, ignorando il resto dell'informazione;
- un'**entità**: una descrizione semplificata di un sistema che enfatizza alcuni dei dettagli o proprietà trascurandone altri.

In programmazione, l'astrazione allude alla distinzione che si fa tra:

- *cosa* (*what*) un programma fa;
- *come* (*how*) viene implementato.

Per l'utente del codice l'essenziale è cosa fa il codice mentre non è interessato ai dettagli implementativi.

Nell'informatica l'astrazione è un concetto importante, in quanto, i sistemi software diventano sempre più complessi e per *padroneggiare la complessità* è necessario concentrarsi solo sui pochi aspetti che più interessano in un certo contesto e ignorarne altri.

L'astrazione permette ai progettisti di sistemi software di risolvere problemi complessi in modo organizzato e gestibile.

Esistono diverse tipologie di astrazione: l'*astrazione funzionale* e l'*astrazione dati*.

1.1 Astrazione funzionale

L'astrazione funzionale si riferisce alla *progettazione del software*, e in particolare alla possibilità di specificare un modulo software che **trasforma** dei dati di input in dati di output nascondendo i dettagli algoritmici della trasformazione. Sinteticamente, un modulo software deve trasformare un input in un output, cioè deve calcolare una funzione. I dettagli della trasformazione non sono visibili al consumatore del modulo, esso conosce solo le corrette convenzioni di chiamata, la *specifica sintattica*, e cosa fa il modulo, la *specifica semantica* e il consumatore deve fidarsi del risultato.

Un esempio di modulo software è il modulo che realizza un operatore per il calcolo del fattoriale. La *specifica sintattica* indica il nome del modulo, il tipo di dato in input e il tipo di risultato, in modo da permettere la corretta chiamata del modulo. Per la funzione del fattoriale:

$$fatt(intero) \rightarrow intero$$

La *specifica semantica* indica la trasformazione operata, cioè la funzione calcolata:

$$fatt(n) = \begin{cases} n(n-1) \cdots 2 \cdot 1 & n \geq 1 \\ 1 & n = 0 \end{cases}$$

Come la trasformazione è calcolata o realizzata non è noto al fruitore del modulo.

1.1.1 Specifiche semantiche

Un modo per specificare la semantica di un modulo è quello di esprimere, mediante due *predicati*, la relazione che lega i dati di ingresso e ai dati di uscita: se il primo predicato, detto *precondizione*, è vero sui dati di ingresso e se il programma termina su quei dati, allora il secondo, detto *postcondizione*, è vero sui dati di uscita.

Queste specifiche semantiche sono dette **assiomatiche**.

Nell'esempio del fattoriale:

- **Precondizione:** $n \in \mathbb{N}$
- **Postcondizione:** $fatt(n) = \begin{cases} n(n-1) \cdots 2 \cdot 1 & n \geq 1 \\ 1 & n = 0 \end{cases}$

Esercizio 1.1.1

Un modulo ha in input un intero x e un vettore $\mathbf{A}$ di n interi e restituisce un intero p e un vettore $\mathbf{B}$ di n interi.

Siano date le seguenti:

- **Precondizione:** $\{n > 0 \wedge \forall i \in [1, n] A[i] \in Z\}$
- **Postcondizione:** $\{\forall i \in [1, n] \exists j \in [1, n] B[i] = A[j] \wedge \forall i \in [1, p - 1] B[i] \leq x \wedge \forall i \in [p + 1, n] B[i] > x\}$

Questa è la specifica semantica del modulo dell'*x partition del quick sort*

1.1.2 Limiti dell'astrazione funzionale

L'astrazione funzionale non permette di progettare, e quindi sviluppare, moduli software *invarianti ai cambiamenti nei dati*, sono invariant solo ai cambiamenti nei processi di trasformazione che operano.

Questo rende difficoltosa la manutenzione delle soluzioni progettate ed è inappropriata per lo sviluppo di soluzioni a problemi complessi.

1.2 Astrazione dati

Alla base dell'*astrazione dati* c'è il principio che *non si può accedere direttamente alla rappresentazione di un dato, qualunque esso sia, ma solo attraverso un insieme di operazioni considerate lecite* e questo è detto **principio dell'astrazione dati**.

Un vantaggio dell'astrazione dati è che un cambiamento nella rappresentazione del dato si ripercuoterà solo sulle operazioni lecite, che potrebbero subire delle modifiche, mentre non inficerà il codice che utilizza il dato astratto.

1.2.1 Information Hiding

In generale un principio di astrazione suggerisce di *occultare l'informazione*, l'**information hiding**, sulla rappresentazione del dato sia perchè non necessaria al fruitore dell'entità astratta e sia perchè la sua rivelazione creerebbe delle inutili dipendenze che comprometterebbero l'invarianza ai cambiamenti.

Il principio dell'astrazione funzionale suggerisce di occultare i dettagli del processo di trasformazione. Il principio dell'astrazione dati identifica nella rappresentazione del dato l'informazione da nascondere.

1.2.2 Incapsulamento

L'**incapsulamento** è una tecnica di progettazione consistente nell'impacchettare una collezione di entità, creandone una barriera concettuale. Come l'astrazione, l'incapsulamento sottintende:

- un **processo**: l'impacchettamento;
- un' **entità**: il pacchetto ottenuto.

Ad essa corrispondono tecniche di programmazione che consentono l'incapsulamento dei dati.

L'incapsulamento non dice come devono essere le "pareti" del pacchetto, che potranno essere:

- **trasparenti**: permettendo di vedere tutto ciò che è stato impacchettato;
- **traslucide**: permettendo di vedere in modo parziale il contenuto;
- **opache**: nascondendo tutto il contenuto del pacchetto.

La combinazione del principio dell'astrazione dati con la tecnica dell'incapsulamento suggerisce che:

1. La rappresentazione del dato va nascosta;
2. L'accesso al dato deve passare solo attraverso operazioni lecite;
3. Le operazioni lecite, che ovviamente devono avere accesso all'informazione sulla rappresentazione del dato, vanno impacchettate con la rappresentazione del dato stesso.

L'astrazione dati ricalca ed estende quella funzionale. Attualmente la possibilità effettuare astrazione di dati è considerata importante almeno quanto quella di definire nuovi operatori con astrazioni funzionali. L'esperienza ha infatti dimostrato che la scelta delle strutture di dati è il primo passo sostanziale per un buon risultato dell'attività di programmazione.

L'astrazione funzionale stimola gli sforzi per evidenziare operazioni ricorrenti o comunque ben caratterizzate all'interno della soluzione di un dato problema.

L'astrazione di dati stimola in più gli sforzi per individuare l'organizzazione dei dati più consone alla soluzione del problema. Si è passati quindi da una progettazione *function centered* ad una *data centered*.

In generale, le astrazioni supportano la separazione dei diversi interessi di:

- **Utenti:** interessati a cosa si astrae (*what*);
- **Implementatori:** interessati a come (*how*) si realizza.

Per questa ragione una definizione di astrazione ha sempre due componenti:

- la **specificità**;
- la **realizzazione**.

Per descrivere una specificità occorre ricorrere a dei *linguaggi di specificità*, che sono diversi dai linguaggi usati per descrivere le realizzazioni delle astrazioni.

1.3 Specificità sintattica e semantica

La specificità potrà poi essere:

- **Sintattica:** stabilisce quali identificatori sono associati all'astrazione;
- **Semantica:** definisce il risultato della computazione inclusa nell'astrazione.

L'efficacia di un'astrazione può essere migliorata mediante l'uso di **parametri** per la comunicazione con l'ambiente esterno. Quando un'astrazione è chiamata, ciascun parametro formale verrà associato in qualche modo al corrispondente argomento.

Anche un'astrazione dati è costituita da una **specificità** e da una **realizzazione**:

- la **specificità** consente di descrivere un nuovo dato e gli operatori che ad esso sono applicabili;
- la **realizzazione** stabilisce come il nuovo dato e i nuovi operatori vengono ricondotti ai dati e agli operatori già disponibili.

Chi intende usare i nuovi dati con i nuovi operatori nella scrittura dei suoi programmi sarà tenuto a conoscere la specificità dell'astrazione dei dati, ma potrà astrarre delle tecniche utilizzate per la realizzazione.

I linguaggi di specificità per astrazione dati più noti sono due:

1. il linguaggio logico-matematico usato nelle asserzioni, chiamate **specifiche assiomatiche**;
2. il linguaggio dell'algebra usato nelle equazioni definite fra gli operatori specificati nel dato astratto, chiamate **specifiche algebriche**.

1.3.1 Specifiche assiomatiche

Un linguaggio formale per la specifica di un dato astratto è costituito dalla notazione logico-matematica delle **asserzioni**. In questo caso si parla di specifica assiomatica.

Una specifica assiomatica è composta da:

- una specifica **sintattica**, detta *signature*, che fornisce:
 - l'elenco dei nomi dei domini e delle operazioni specifiche del tipo;
 - i domini di partenza e di arrivo per ogni nome di operatore.
- una specifica **semantica** che associa:
 - un insieme ad ogni tipo introdotto nella specifica sintattica;
 - una funzione ad ogni nome di operatore, esplicitando le condizioni sui domini di partenza e di arrivo:
 - * *precondizione*, che definisce quando l'operatore è applicabile;
 - * *postcondizione*, che stabilisce la relazione tra argomenti e risultato.

Limiti delle specifiche assiomatiche Si noti che il metodo di specifica assiomatica è preciso nella definizione della specifica sintattica, mentre è piuttosto informale per altri aspetti, infatti a volte si ricorre al linguaggio naturale per semplicità. Pertanto esso non consente di caratterizzare precisamente un dato astratto. In particolare, non consente di definire i valori che possono essere generati mediante l'applicazione di operatori, e non consente di stabilire quando l'applicazione di diverse sequenze di operatori porta al medesimo valore. Questo problema è superato dalle specifiche algebriche.

1.3.2 Specifica algebriche

Le specifiche algebriche si basano sull'algebra, piuttosto che sulla logica. Essenzialmente definiscono un dato astratto come un'**algebra eterogenea**, ovvero come una collezione di diversi insiemi su cui sono definite diverse operazioni. Le algebre tradizionali sono omogenee. Un'algebra omogenea consiste in un unico insieme e diverse operazioni.

Ad esempio, l'insieme $\mathbb{Z}$ con le operazioni di addizione e moltiplicazione è un

algebra omogenea.

Invece, se consideriamo un alfabeto Σ e consideriamo Σ^* l'insieme di tutte le stringhe, inclusa quella vuota, costruite con i simboli di Σ .

Σ^* con le operazioni di concatenazione e calcolo della lunghezza non sono un'algebra omogenea, visto che il codominio dell'operazione di calcolo della lunghezza è $\mathbb{N}$ e non Σ^* . Quindi, quest'algebra consiste di due insiemi, stringhe e interi, su cui sono definite le operazioni di concatenamento e lunghezza delle stringhe.

Una specifica algebrica consiste di tre parti:

1. **Sintattica:** elenca i nomi del tipo, le sue operazioni e il tipo degli argomenti delle operazioni. Se un'operazione è una funzione allora è specificato il codominio della funzione;
2. **Semantica:** consiste di un insieme di equazioni algebriche che descrivono in modo indipendente dalla rappresentazione le proprietà delle operazioni;
3. **Di restrizione:** stabilisce varie condizioni che devono essere soddisfatte o prima che siano applicate le operazioni o dopo che esse siano state completate.

A volte si possono trovare le specifiche di restrizione in quelle semantiche.

Uno degli aspetti più interessanti delle specifiche algebriche è la semplicità del linguaggio di specifica rispetto ai linguaggi di programmazione procedurale. Infatti, il linguaggio di specifica consiste di solo cinque primitive:

1. composizione funzionale;
2. relazione di equaglianza;
3. la costante *true*;
4. la costante *false*;
5. un numero illimitato di variabili libere.

La funzione *if then else* può essere facilmente descritta dalle seguenti equazioni:

$$\text{if then else}(\text{true}, q, r) = q$$

$$\text{if then else}(\text{false}, q, r) = r$$

Questa funzione è così importante che si assume già data come operatore infisso:

$$\text{if then } q \text{ else } r$$

Inoltre si assume che sono predefiniti i valori interi e booleani

Esempio : Specifica algebrica di Pila

- **Specifica sintattica**

sorts: stack, item, boolean

operations:

- $\text{newstack}() \rightarrow \text{stack}$
- $\text{push}(\text{stack}, \text{item}) \rightarrow \text{stack}$
- $\text{pop}(\text{stack}) \rightarrow \text{stack}$
- $\text{top}(\text{stack}) \rightarrow \text{item}$
- $\text{isnew}(\text{stack}) \rightarrow \text{boolean}$

Ogni insieme è detto un *sort* dell'algebra eterogenea. I *sort* item e boolean sono *ausiliari* alla definizione di stack.

- **Specifica semantica**

declare *stk*: stack, *i*: item

- $\text{pop}(\text{push}(\text{stk}, i)) = \text{stk}$
- $\text{top}(\text{push}(\text{stk}, i)) = i$
- $\text{isnew}(\text{newstack}) = \text{true}$
- $\text{isnew}(\text{push}(\text{stk}, i)) = \text{false}$

- **Specifica di restrizione**

- $\text{pop}(\text{newstack}) = \text{error}$
- $\text{top}(\text{newstack}) = \text{error}$

dove *error* è un elemento speciale indefinito.

Ovviamente avremmo potuto scrivere molte altre equazioni, come:

$$\text{isnew}(\text{pop}(\text{push}(\text{newstack}, i))) = \text{true}$$

che evidenzia come il predicato *isnew* sia vero anche per quelli stack ottenuti inserendo un generico elemento *i* in un **nuovo** stack e poi rimuovendolo.

Tuttavia questa equazione è ridondante, poichè è ricavabile da altre scritte in precedenza. Infatti, grazie alla prima e alla quarta equazione possiamo scrivere $\text{isnewstack}(\text{pop}(\text{push}(\text{newstack}, i))) = \text{isnew}(\text{newstack}) = \text{true}$.

Nelle specifiche semantiche è importante indicare l'insieme minimale di equazioni, dette *assiomi*, a partire dalle quali possiamo derivare tutte le altre. Le specifiche semantiche si diranno **incomplete** se non permetteranno di derivare tutte le verità desiderate dell'algebra specificate.

Le specifiche semantiche si diranno **inconsistenti** se permetteranno di derivare delle equazioni indesiderate, cioè considerate false. Le specifiche semantiche si diranno **ridondanti** se alcune delle equazioni sono ricavabili dalle altre.

Scrivere delle specifiche semantiche complete, consistenti e non ridondanti può non essere un compito semplice. Per questo conviene introdurre una **metodologia**, che si basa sulla distinzione degli operatori di un dato astratto in:

- **costruttori**, che creano o istanziano il dato astratto;
- **osservazioni**, che ritrovano informazioni sul dato astratto.

Il comportamento di un'astrazione dati può essere specificata riportando il valore di ciascuna osservazione applicata a ciascun costruttore. Questa informazione è organizzata in modo naturale in una matrice, con i costruttori lungo una dimensione e le osservazioni lungo l'altra.

Un esempio di costruttori e osservazioni per la pila è il seguente.

<i>osservazioni</i>	<i>Costruttore di stk'</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>pop(stk')</i>	error	stk
<i>top(stk')</i>	error	i
<i>isnew(stk')</i>	true	false

Tutte le osservazioni viste finora sono unarie, nel senso che esse osservano un singolo valore del dato astratto, ma spesso, è necessario disporre di osservazioni più complesse. Per confrontare due valori è necessario osservare due istanze del dato astratto e questo complica la specifica, perchè il valore dell'osservazione dev'essere definito per tutte le **combinazioni di costruttori** possibili per i valori astratti che si devono confrontare.

Esempio : Predicato *equal*

Consideriamo il predicato *equal(l, m)* che è vero solo se le due pile contengono gli stessi elementi nello stesso ordine.

<i>Costruttore di m</i>	<i>Costruttore di l</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>newstack</i>	true	false
<i>push(stk', i')</i>	false	i=i' and equal(stk,stk')

Questa tabella può essere vista come l'aggiunta di una terza dimensione alla tabella di base per le osservazioni unarie.

Questa terza dimensione può essere rimossa andando a classificare esplicitamente solo il primo argomento dell'osservazione e referenziando in modo astratto il secondo argomento mediante una *variabile libera*.

<i>osservazione</i>	<i>Costruttore di stk'</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>equal(stk', m)</i>	isnew(m)	not isnew(m) and i=top(m) and equal(stk,pop(m))

Esempio : Specifica algebrica della coda

- **Specifica sintattica**

sorts: queue, item, boolean

operations:

- newq() → queue
- addq(queue, item) → queue
- deleteq(queue) → queue
- frontq(queue) → item
- isnewq(queue) → boolean

- **Specifica semantica**

declare q : *queue*, i : *item*

- $\text{deleteq}(\text{addq}(q, i)) = \text{if } \text{isnewq}(q) \text{ then } \text{newq} \text{ else } \text{addq}(\text{deleteq}(q), i)$
- $\text{frontq}(\text{addq}(q, i)) = \text{if } \text{isnewq}(q) \text{ then } i \text{ else } \text{frontq}(q)$
- $\text{isnew}(\text{newq}) = \text{true}$
- $\text{isnew}(\text{addq}(q, i)) = \text{false}$

- **Specifica di restrizione**

- $\text{deleteq}(\text{newq}) = \text{error}$
- $\text{frontq}(\text{newq}) = \text{error}$

Se volessi rappresentare costruttori e osservazioni mediante una tabella, il risultato è il seguente.

osservazioni	<i>Costruttore di q'</i>	
	<i>newq</i>	<i>addq(q, i)</i>
<i>isnewq(q')</i>	true	false
<i>frontq(q')</i>	error	if <i>isnewq(q)</i> then <i>i</i> else <i>frontq(q)</i>
<i>deleteq(q')</i>	error	if <i>isnewq(q)</i> then <i>newq</i> else <i>addq(deleteq(q), i)</i>

Capitolo 2

L'astrazione nella programmazione

È possibile costruire astrazioni su una qualunque classe sintattica, purchè le frasi di quella classe specifichino un qualche tipo di computazione.

Di seguito si esaminerà l'applicazione del principio di astrazione in sei classi sintattiche, in particolare:

1. Espressione → **astrazione di funzione**;
2. Comando → **astrazione di procedura**;
3. Controllo di sequenza → **astrazione di controllo**;
4. Accesso a un'area di memoria → **astrazione di selettore**;
5. Definizione di un dato → **astrazione di tipo**;
6. Dichiarazione → **astrazione generica**.

Tutte queste classi sintattiche sottointendono una computazione, infatti:

- L'astrazione di *funzione* include un'espressione da valutare.
- L'astrazione di una *procedura* include un comando da seguire.
- L'astrazione di *controllo* include delle espressioni di controllo dell'ordine di esecuzione delle istruzioni.
- L'astrazione di *selettore* include il calcolo dell'accesso ad una variabile.

- L'astrazione di *tipo* include un gruppo di operatori che definiscono implicitamente un insieme di valori.
- L'astrazione *generica* include una frase che sarà elaborata per produrre legami.

2.1 Astrazione di funzione

Un'astrazione di funzioni include un'espressione da valutare, e quando chiamata, darà un valore come risultato.

Un'astrazione di funzione è specificata mediante una definizione di funzione del tipo:

function $I(FP_1; \dots; FP_n)$ is E

dove I è un identificatore, $FP_1; \dots; FP_n$ sono parametri formali ed E è l'espressione da valutare.

In questo modo si lega I ad un'entità, l'astrazione di funzione, che ha la proprietà di dare un risultato ogni qualvolta che è chiamata con argomenti appropriati.

Una chiamata di funzione, $I(AP_1; \dots; AP_n)$ dove i parametri effettivi, $AP_1; \dots; AP_n$ determinano gli argomenti, ha due punti di vista:

1. Punto di vista dell'*utente*: la chiamata di una funzione trasforma gli argomenti in un risultato;
2. Punto di vista dell'*implementatore*: la chiamata valuta E , avendo precedentemente vincolato i parametri formali agli argomenti corrispondenti.
L'algoritmo codificato in E è di interesse per l'implementatore.

2.1.1 Anomalie di progetto dei linguaggi

Se l'astrazione di funzione include un'espressione E da valutare, è naturale attendersi che il corpo della funzione non includa dei comandi, come assegnazioni, istruzioni di salto, iterazioni, e altro, il cui effetto è quello di cambiare lo stato di un sistema, non di produrre valori.

Non sempre ciò accade, come si può osservare in questo esempio di funzione scritto in Pascal.

```

1  function power(x: real; n: integer) : real
2  begin
3      if n = 1 then power := x

```

```

4     else power := x * power(x, n - 1)
5 end
6

```

Nel corpo di questa funzione compaiono i comandi di assegnazione. Non è proprio possibile farne a meno, perchè per poter restituire un valore, in Pascal è necessario assegnare un valore ad una pseudo variabile, che ha lo stesso nome della funzione.

Il corpo della funzione è sintatticamente un comando, ma semanticamente è un'espressione, questa funzione, infatti, può essere invocata solo alla destra di operazioni di assegnazione.

Non tutti i linguaggi di programmazione includono comandi nel corpo di funzioni, infatti, nel linguaggio funzionale ML la funzione vista prima diventa:

```

1 function power(x: real; n: int) is
2   if n = 1 then x
3   else x * power(x, n-1)
4

```

Nel corpo della funzione definita in ML compare un'espressione condizionale la cui valutazione non modifica lo stato del sistema. Il motivo per il quale altri linguaggi di programmazione, come Pascal, Ada, o altro, permettono di avere comandi nel corpo di funzione è quello di poter sfruttare la potenza espressiva dell'assegnazione e dell'*iterazione* nella computazione di risultati. Diversamente saremmo costretti a esprimere ricorsivamente le espressioni da valutare. E la ricorsione, com'è noto, può essere fonte di inefficienze nell'uso delle risorse di calcolo.

Quindi, per ragioni di efficienza, molti linguaggi permettono di avere comandi nel corpo di funzioni e si lascia al programmatore il compito di utilizzare correttamente i comandi in modo da evitare che la funzione possa avere effetti collaterali, *side effect*, oltre quello di calcolare un valore.

Le funzioni in quanto astrazioni di espressioni, possono comparire ovunque occorra un'espressione. Pertanto, esse compaiono alla destra di operazioni di assegnazioni, ma anche al posto di parametri effettivi nelle chiamate di altre funzioni, laddove il valore potrebbe essere calcolato mediante un'espressione.

$$y := f(y, \text{power}(x, 2))$$

In molti linguaggi di programmazione è possibile definire dei parametri formali che sono riferimento ad una funzione.

```

1 function sommatoria(function F(R: real; M: integer): real; X: real; N:
integer) : real;

```



```

2   var I: integer;
3   var Sum: real;
4   begin
5       Sum := 0;
6       for I := 1 to N do Sum := Sum + F(x, I);
7       Sommatoria := Sum
8   end
9

```

Se la funzione F è la funzione *power* definita in precedenza, allora:

$$Y := Sommatoria(power, x, n)$$

calcola la sommatoria $\sum_{i=1}^n x^i$.

In alcuni linguaggi si separa il concetto di astrazione di funzione da quello di legame a un identificatore. Questo permette di passare come parametri delle funzioni anonime. Infine, le funzioni possono essere il risultato di valutazioni di espressioni o possono essere assegnate a variabili:

```

1   val cube = fn(x: real) => x * x * x

```

Riepilogando, in alcuni linguaggi di programmazione, le funzioni sono di *terza classe*, cioè possono essere solo chiamate, in altri sono di *seconda classe*, cioè possono essere passate come argomenti, mentre in altri linguaggi di programmazione esse sono di *prima classe* in quanto possono essere anche restituite come risultato della chiamata di altre funzioni o possono essere assegnate come valore a una variabile.

Cittadini di prima classe In generale, in programmazione una qualunque entità si dice *cittadino di prima classe* quando non è soggetta a restrizioni nel suo utilizzo. A tal proposito, osserviamo che i valori possono essere:

- *Denotabili*, se possono essere associati ad un nome;
- *Esprimibili*, se possono essere il risultato di un'espressione complessa;
- *Memorizzabili*, se possono essere memorizzati in una variabile.

2.2 Astrazione di procedura

Un'astrazione di procedura include un comando da eseguire, e quando chiamata, aggiornerà le variabili che rappresentano lo stato del sistema. Un'astrazione di

procedura è specificata mediante una definizione di procedura, del tipo:

$$\text{procedure } I(\text{FP}_1; \dots; \text{FP}_n) \text{ is } C$$

dove I è un identificatore, $\text{FP}_1; \dots; \text{FP}_n$ sono parametri formali, e C è il blocco di comandi da eseguire.

In questo modo si lega I all'astrazione di procedura, che gode della proprietà di cambiare lo stato del sistema quando chiamata con argomenti appropriati.

Data una chiamata di procedura, $I(\text{AP}_1; \dots; \text{AP}_n)$ dove $\text{AP}_1; \dots; \text{AP}_n$ sono i parametri effettivi, il punto di vista dell'*utente* è che la chiamata aggiornerà lo stato del sistema in modo dipendente dai parametri, mentre il punto di vista dell'*implementatore* è che la chiamata consentirà l'esecuzione del corpo di procedura C , avendo precedentemente vincolato i parametri formali agli argomenti corrispondenti. L'algoritmo in C è di interesse solo per l'implementatore.

In Pascal e in C le procedure sono *cittadini di seconda classe*.

L'astrazione di funzione o di procedura sono due tecniche di programmazione di supporto all'astrazione funzionale, intesa come tecnica di progettazione del software seconda la quale occorre distinguere la specifica di un operatore dalla sua realizzazione.

Quale sia la tecnica di programmazione, se di funzione o di procedura, più opportuna da adottare dipende da:

- **Tipo di operatore progettato:**

- ha effetti collaterali, come accade nell'astrazione di procedura;
- non ha effetti collaterali, come accade nell'astrazione di funzione.

- **Limiti imposti dal linguaggio di programmazione.**

Ad esempio, l'operatore di *bubblesort* non modifica l'array passato in ingresso, ma ne restituisce uno ordinato, pertanto sarebbe più opportuno implementarlo ricorrendo ad un'astrazione di funzione. Tuttavia, non tutti i linguaggi di programmazione consentono di restituire come un tipo di dato complesso e quindi l'operatore *bubblesort* dovrà essere realizzato mediante un'astrazione di procedura.